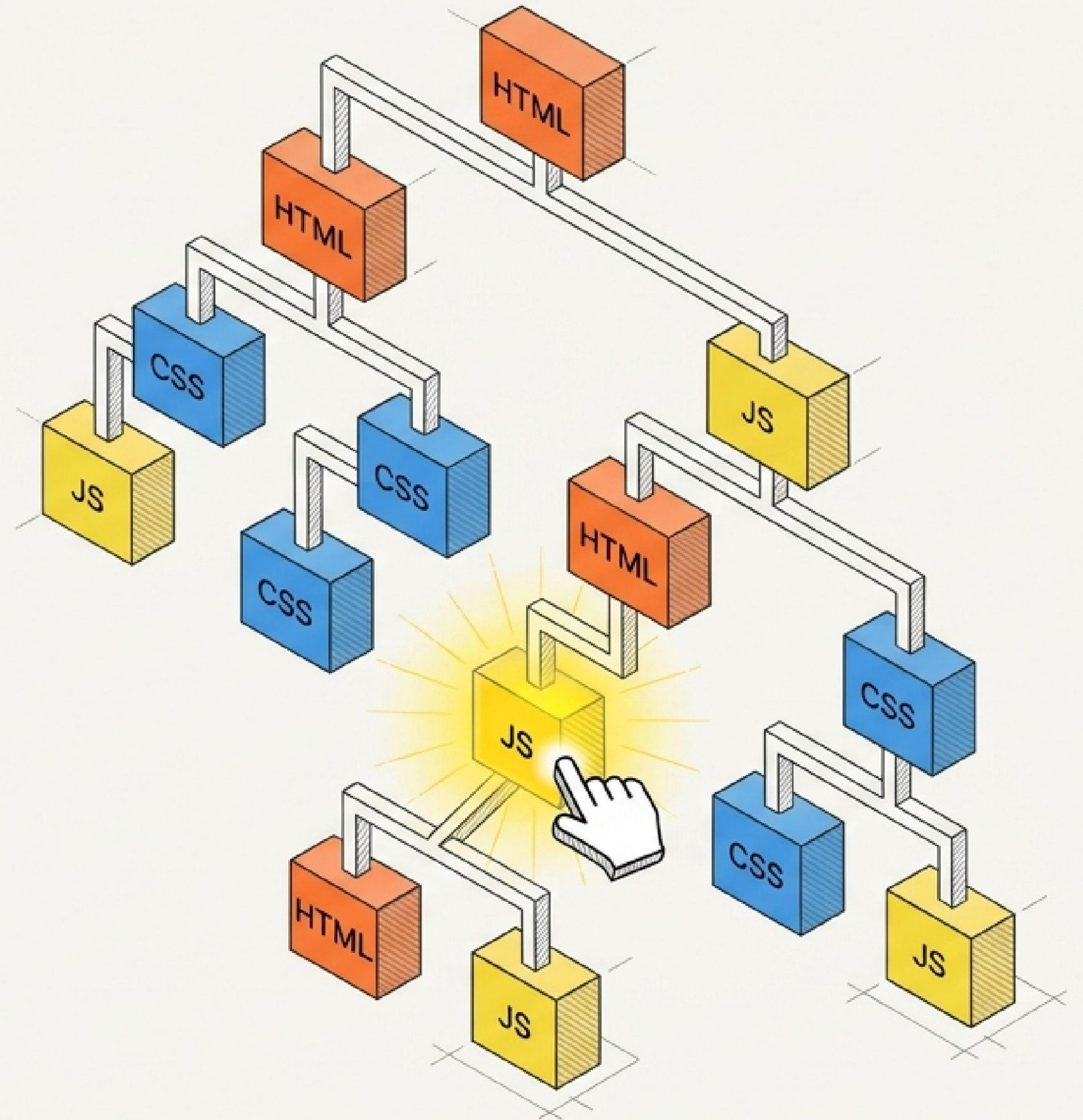


El DOM: La Màgia Interactiva de JavaScript

Guia visual i pràctica per dominar el Model d'Objectes del Document



Què és exactament el DOM?

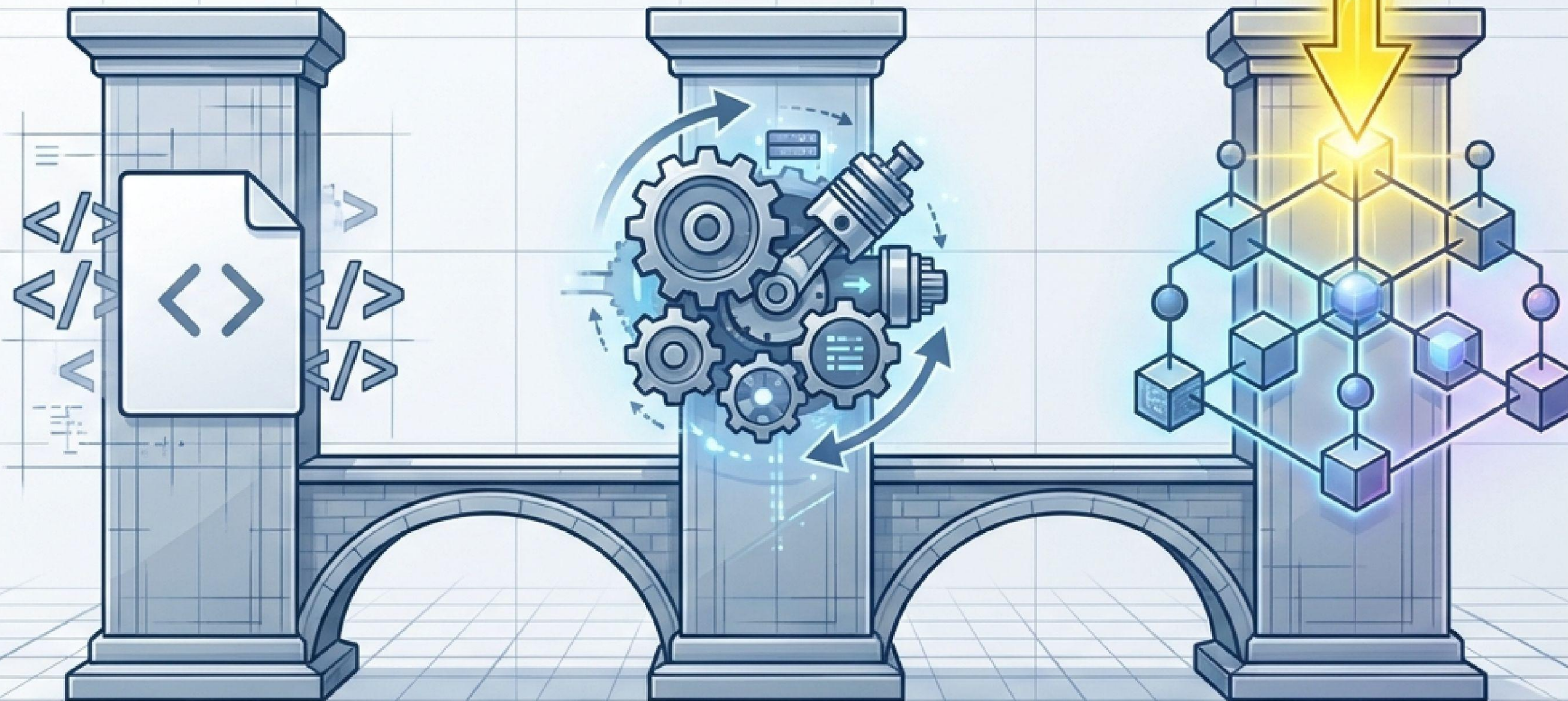
El pont entre el teu codi i el navegador

- Document Object Model.
- Una interfície de programació estandarditzada pel W3C.
- Transforma el text estàtic de l'HTML en un conjunt d'objectes vius que JavaScript pot llegir, alterar i esborrar en temps real.

JS JavaScript



JavaScript modifica el DOM en memòria, no el fitxer HTML original.



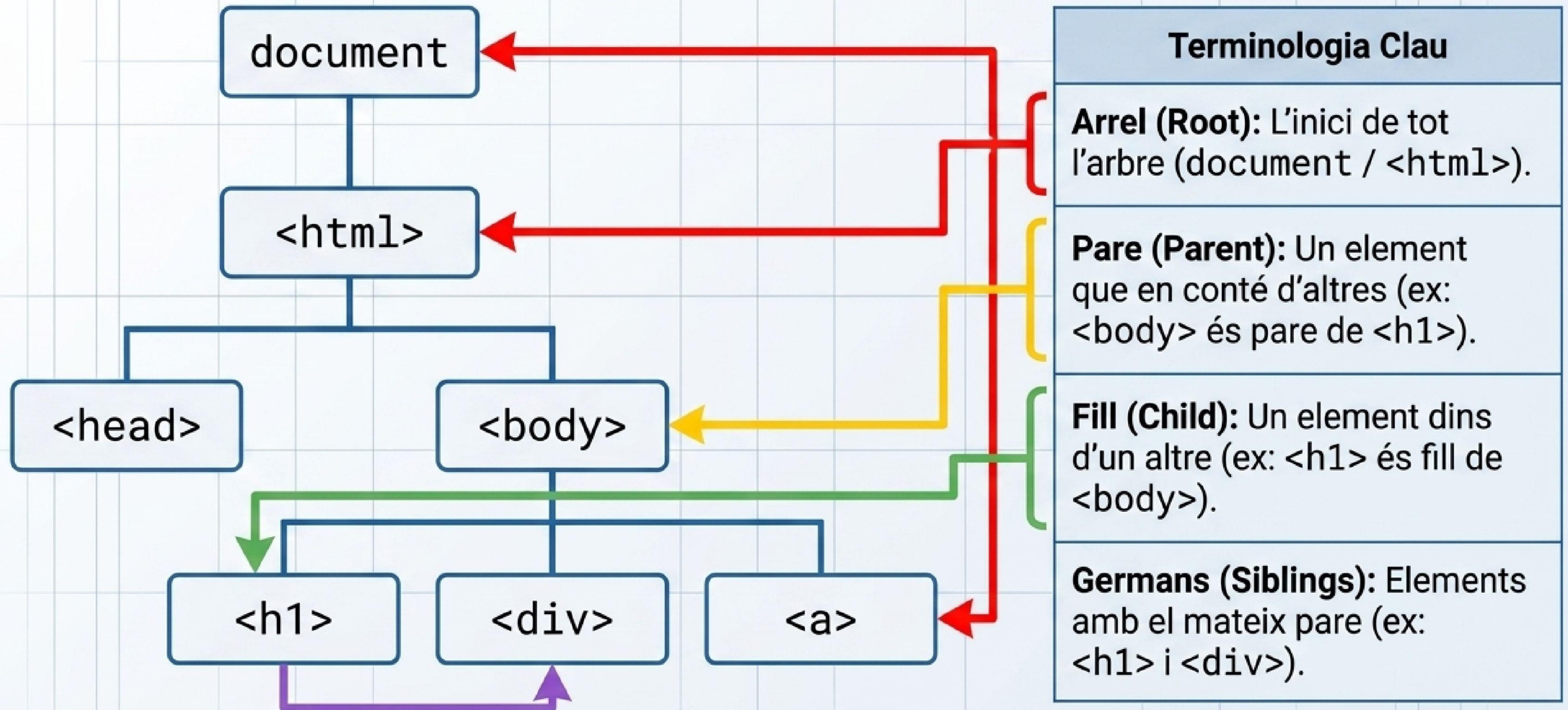
1. HTML
(L'Esquelet)

2. El Navegador
(El Traductor)

3. El DOM
(L'Edifici)

L'Anatomia: L'Arbre del DOM

Com s'organitzen els elements en memòria



Tot és un Node

Els blocs de construcció de l'arbre



En el DOM, cada peça és un **Node** independent amb propietats úniques:



1. Nodes d'Element: Representen les etiquetes HTML. Són l'estructura principal.



2. Nodes d'Atribut: Informació extra de l'etiqueta (classes, IDs, src).

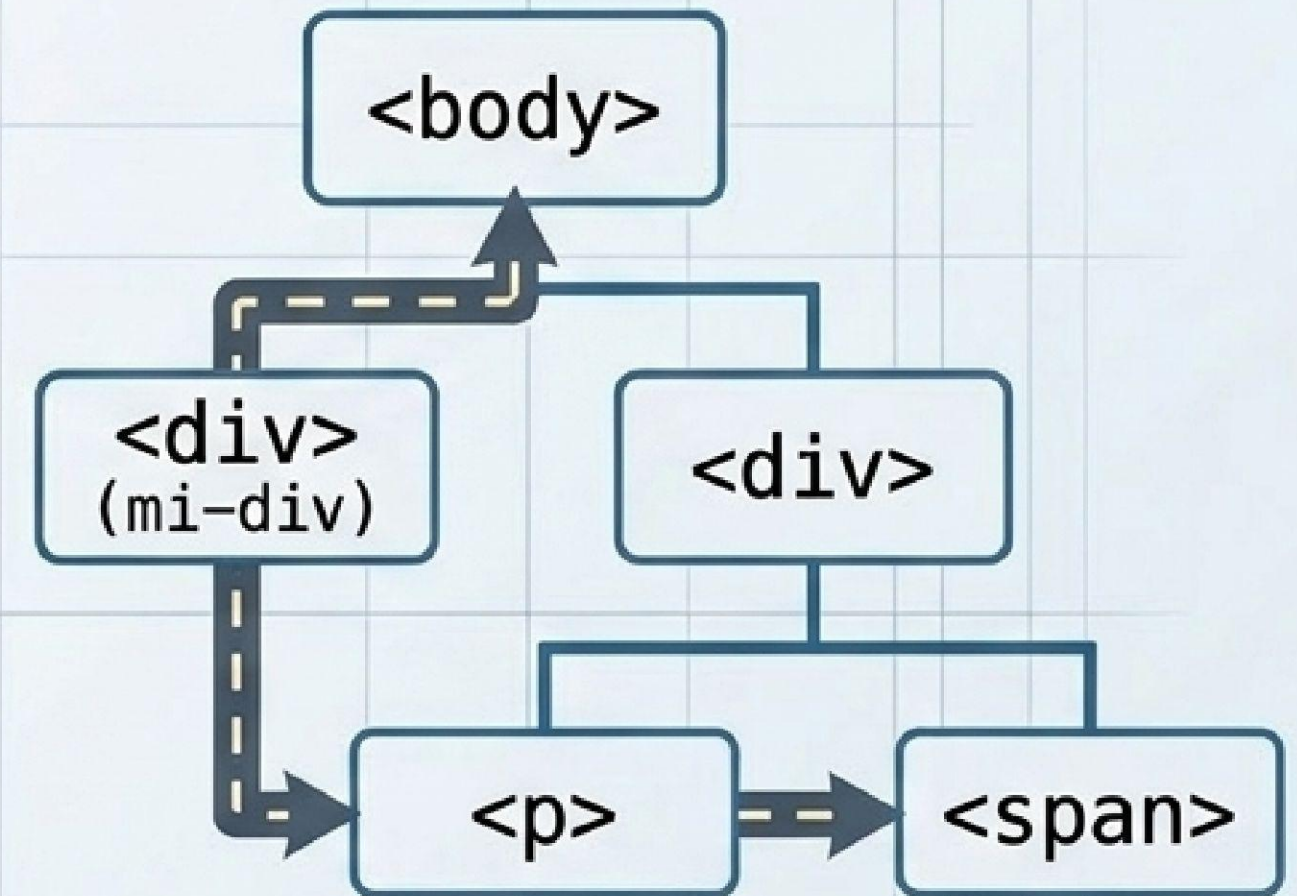


3. Nodes de Text: El contingut real. Alerta: Els espais en blanc i salts de línia a l'HTML també es converteixen en nodes de text ocults!

Navegant per l'Arbre Manualment

Com moure's entre nodes relacionats

```
1 const element = document.getElementById("mi-div");
2
3 // Navegació relativa:
4 const pare = element.parentNode;
5 const fills = element.children; // Només elements
6 const germaSeguent = element.nextElementSibling;
```



- **HTMLCollection** vs **NodeList**: **children** retorna només elements HTML. **childNodes** retorna tots els nodes (inclosos els espais en blanc invisibles).
- **Consell d'Arquitecte**: Moure's node a node pot trencar el codi si afegeixes un espai a l'HTML. És més segur seleccionar directament!

L'Arsenal: Selecció d'Elements

Com atrapar exactament el que busques

Mètode	Què utilitza?	Què retorna?	Exemple
<code>getElementById()</code>	ID únic	1 Element	<code>document.getElementById('logo')</code>
<code>querySelector()</code>	Selector CSS	1r Element	<code>document.querySelector('.boto')</code>
<code>querySelectorAll()</code>	Selector CSS	NodeList estètica	<code>document.querySelectorAll('p')</code>
<code>getElementsByClassName()</code>	Classe	HTMLCollection	<code>document.getElementsByClassName('carta')</code>

Nota: `querySelector` i `querySelectorAll` són els mètodes moderns més versàtils perquè utilitzen exactament la mateixa sintaxi que ja coneixes del teu CSS.

Modificant el Contingut

Donant vida a les paraules

el.innerHTML

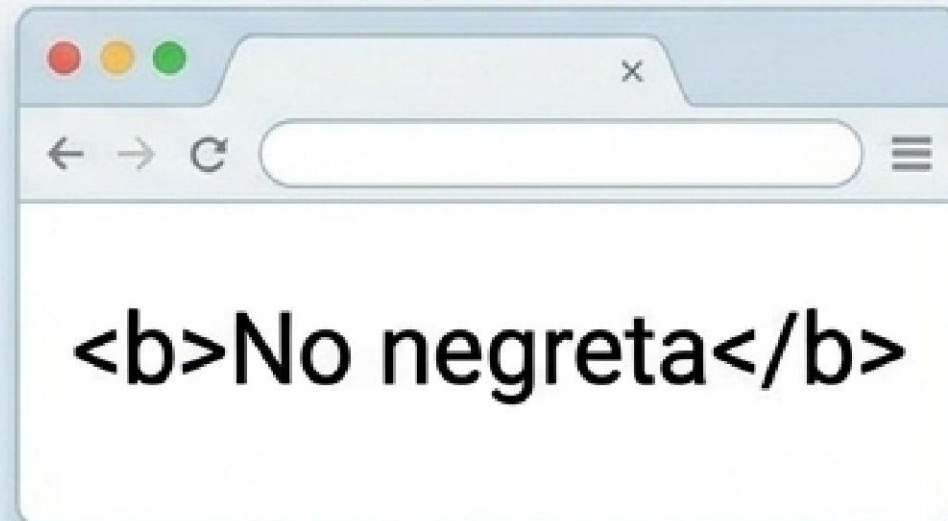
```
1 el.innerHTML =  
2   '<b>Negreta</b>'
```



⚠ Llegeix o insereix HTML real. (Risc de seguretat XSS si s'usen dades d'usuaris).

el.textContent

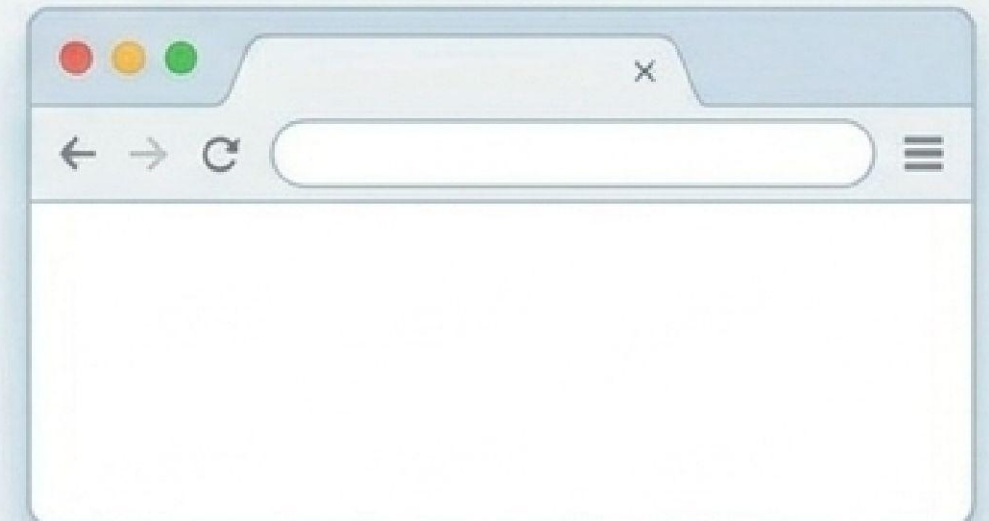
```
1 el.textContent =  
2   '<b>No negreta</b>'
```



Insereix només text, ignorant etiquetes HTML. Llegeix tot el text, inclòs el que està amagat per CSS.

el.innerText

```
1 el.innerText  
2
```



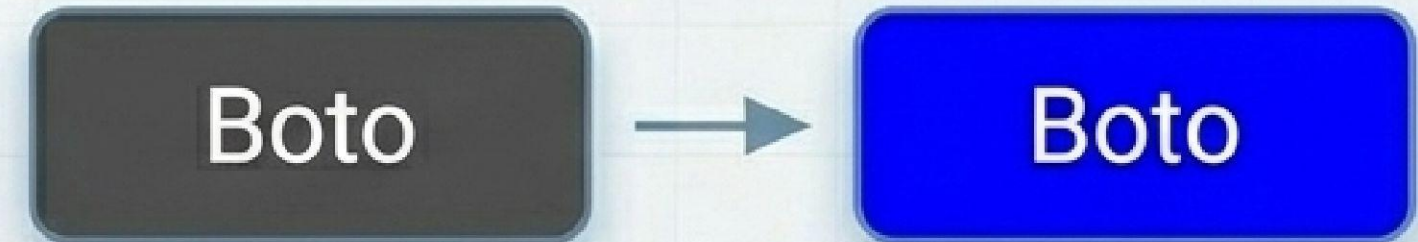
Retorna exclusivament el text visible per l'usuari a la pantalla.

Modificant l'Estètica (Estils i Classes)

Canvis visuals dinàmics

Opció 1: Estils en línia (Propietat style)

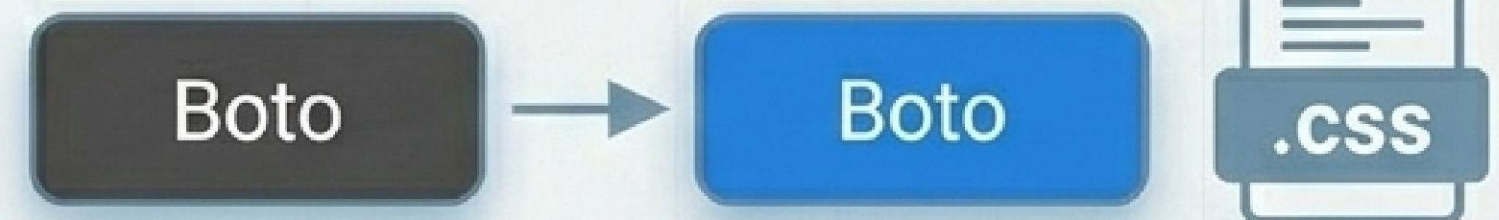
```
1 boto.style.backgroundColor = 'blue';  
2
```



Canvia propietats directament (usa camelCase en comptes de guions).

Opció 2: Manipulació de Classes (classList) - Recomanat!

```
1 boto.classList.add('actiu');  
2 boto.classList.remove('ocult');  
3 boto.classList.toggle('mode-oscuro');
```



Mantén el disseny al CSS i canvia l'estat des de JavaScript. L'alternador (toggle) és intel·ligent.

Manipulant Atributs i Formularis

Controlant el comportament ocult

1. Atributs HTML (src, alt, href):

```
1 const imatge = document.querySelector('img');  
2 imatge.setAttribute('src', 'nova-imatge.jpg');  
3 imatge.removeAttribute('class');
```

Els atributs personalitzats (data-*) s'accedeixen amb .dataset



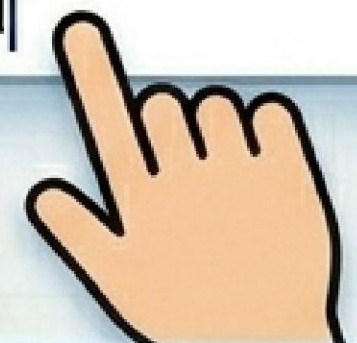
2. Dades d'Usuari (Formularis):

```
1 const input = document.getElementById('nom');  
2 console.log(input.value); // Obté el que l'usuari ha escrit  
3 input.value = ''; // Neteja el camp de text
```

Captura i manipula valors d'entrada d'usuari per a interactivitat dinàmica.

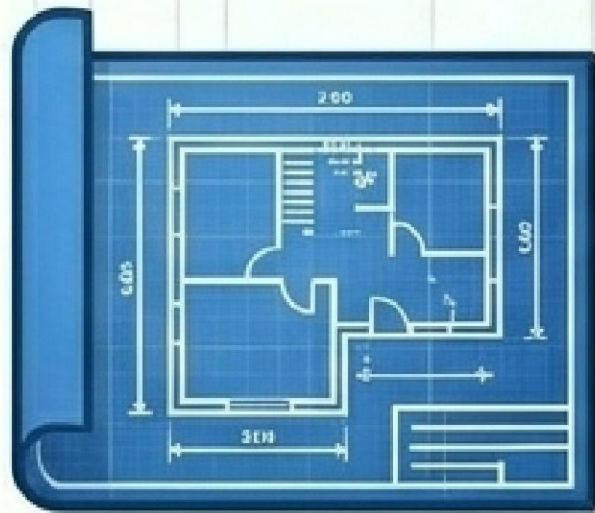
<input>

Maria|

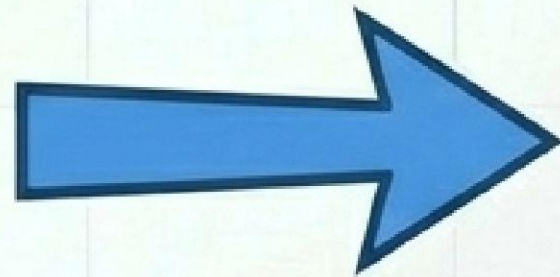


Creació d'Elements (La Fàbrica del DOM)

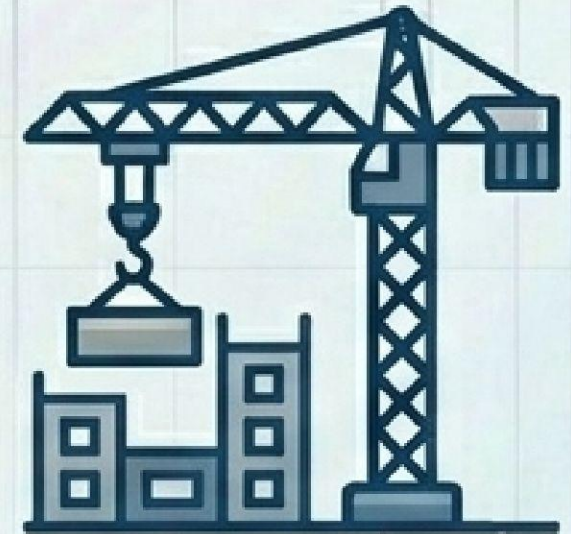
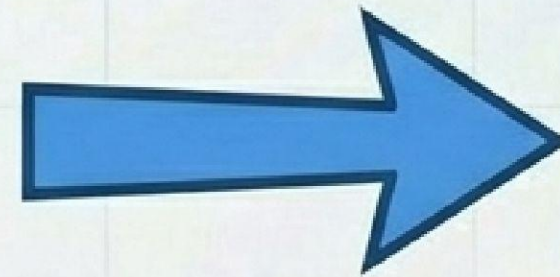
Com construir i inserir nous elements des de zero



1. Crear
(createElement)



2. Personalitzar
(textContent / classList)



3. Inserir
(appendChild)

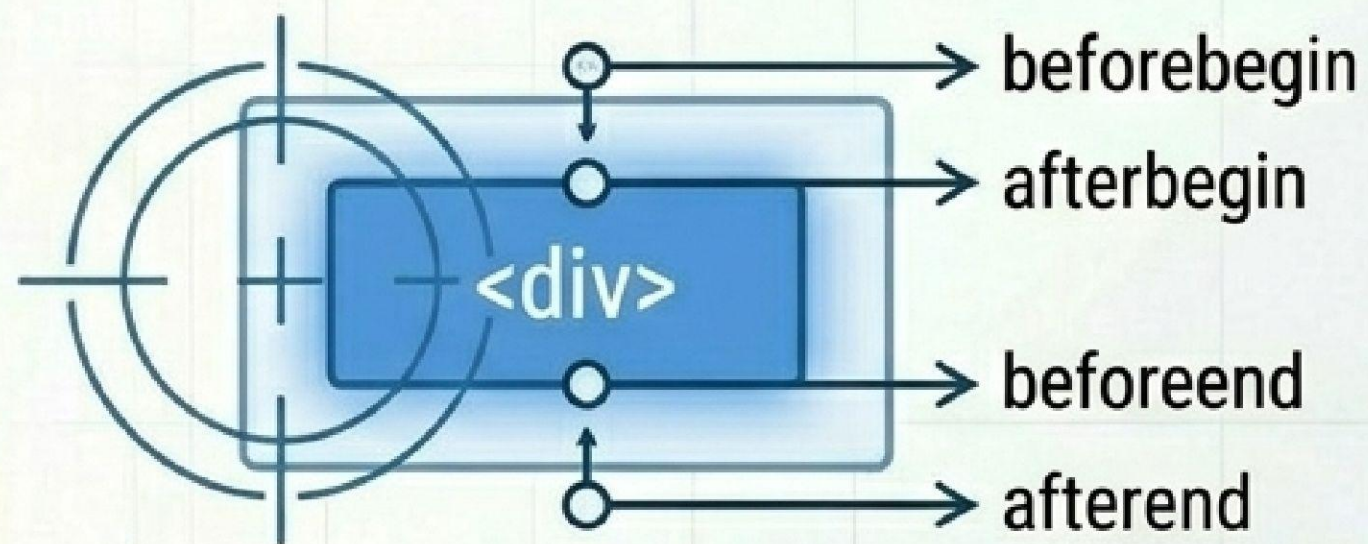
El Codi de Construcció:

```
1 // 1. Crear l'element en memòria (encara invisible)
2 const nouParagraf = document.createElement('p');
3 // 2. Afegir contingut i estils
4 nouParagraf.textContent = 'Això és un text nou!';
5 nouParagraf.classList.add('destacat');
6 // 3. Connectar-lo a l'arbre del DOM (es fa visible)
7 document.body.appendChild(nouParagraf);
```


Inserció Precisa i Neteja

Col·locació exacta i eliminació de nodes

Col·locació Quirúrgica:



```
1 element.insertAdjacentHTML('beforebegin', '<h2>Títol</h2>');
```

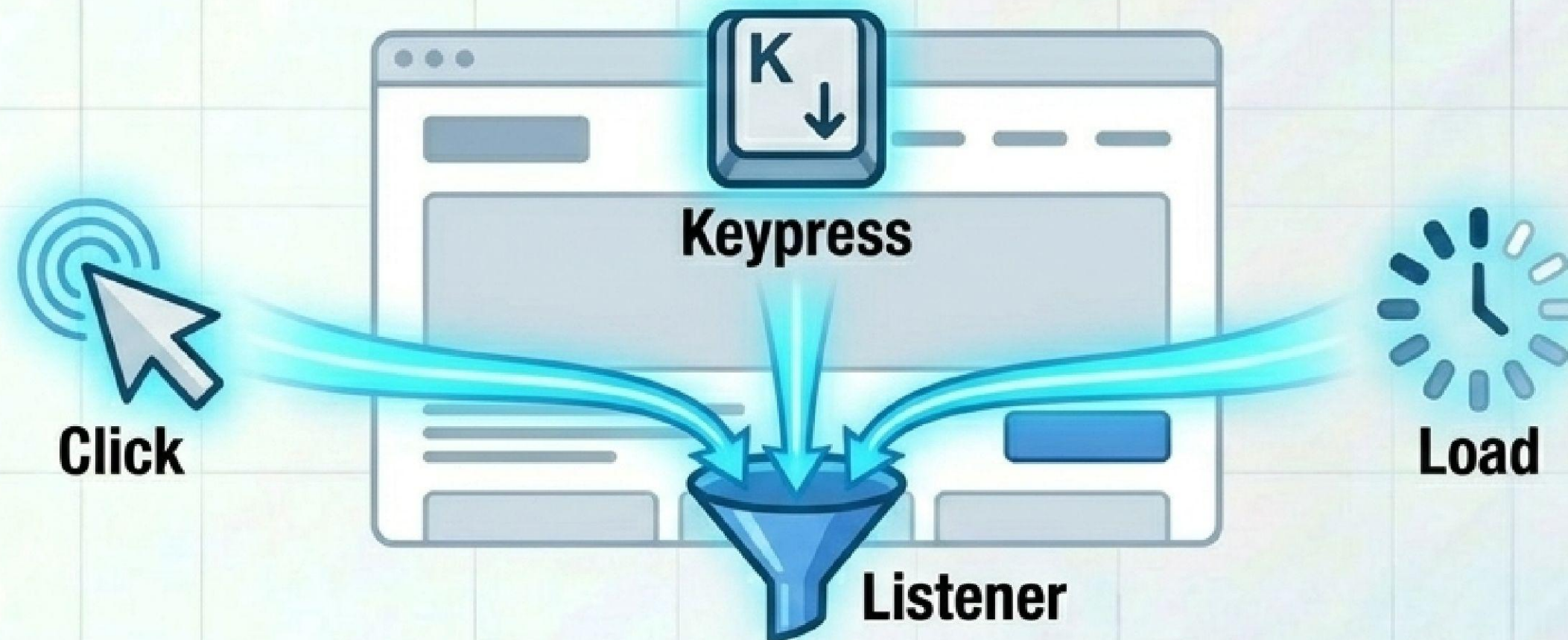
Esborrar Elements:



```
1 const elementAEsborrar = document.querySelector('.anunci');  
2  
3 // Mètode modern (S'elimina a si mateix):  
4 elementAEsborrar.remove();  
5  
6 // Mètode clàssic (Requereix el pare):  
7 pare.removeChild(elementAEsborrar);
```


Interactivitat: Els Esdeveniments (Events)

Escotar i reaccionar a les accions de l'usuari



Per fer una pàgina interactiva, necessitem “escotar” el que fa l'usuari fent servir **addEventListener**.

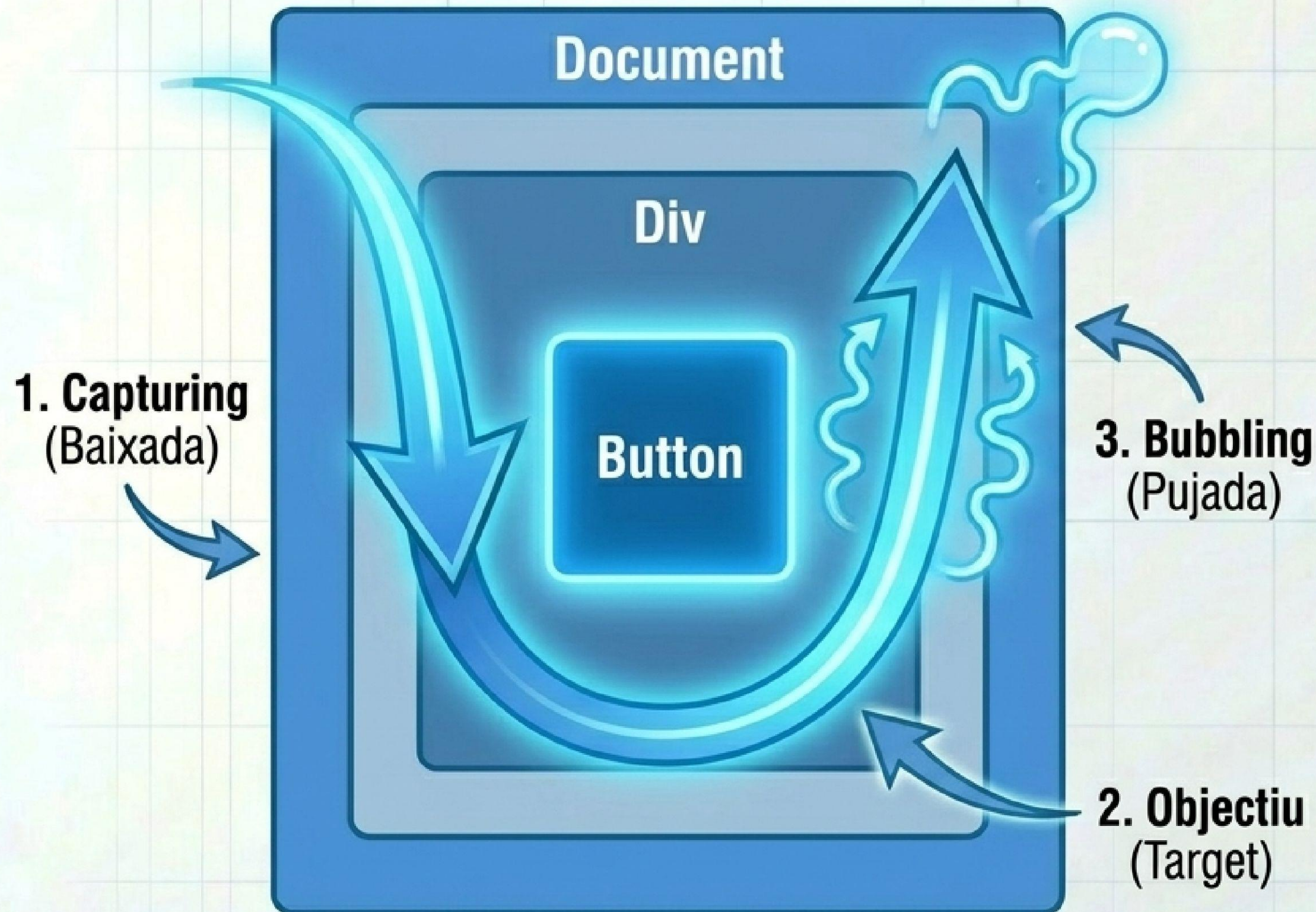
La Recepta Màgica:

```
const boto = document.querySelector('#meuBoto'); // Element + Mètode + (Tipus d'Event, Funció a executar)
boto.addEventListener('click', function(event) {
  event.preventDefault(); // Evita recarregar formularis
  alert('Has fet clic al botó!');
});
```

Esdeveniments comuns: click, mouseover, keydown, submit, DOMContentLoaded.

El Cicle de Vida d'un Esdeveniment

Capturing i Bubbling (El Bombolleig)

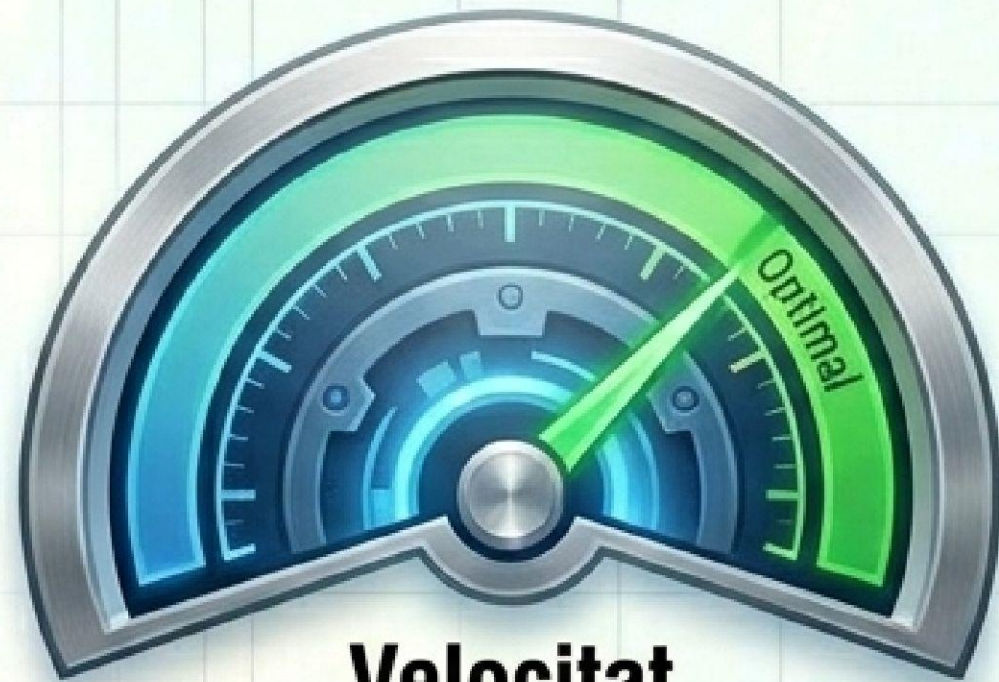


Quan fas clic a un botó dins d'un **div**, l'esdeveniment viatja per l'arbre:

- **Capturing:** Baixa des de l'arrel fins a l'element.
- **Bubbling (Bombolleig):** L'esdeveniment 'rebota' i puja a través de tots els seus pares.
- **El Control:** Usa `event.stopPropagation()` per evitar que l'esdeveniment pugi. Això permet la Delegació d'Esdeveniments (posar un sol listener al pare per controlar molts fills).

Rendiment i Bones Pràctiques

Com ser un arquitecte eficient



Velocitat



Desa les referències (Cache): No busquis el mateix element dins d'un bucle. Cerca'l un cop i desa'l en una variable.



Memòria



Espera a la càrrega: Assegura't que l'HTML existeix abans d'actuar usant l'esdeveniment DOMContentLoaded.



Estabilitat



Evita el Reflow i Repaint: Cada cop que canvis l'estructura o el color, el navegador recalcula el disseny. Agrupa els canvis en blocs.



El Pas a Frameworks: Per a aplicacions massives, llibreries com React o Vue utilitzen un DOM Virtual per optimitzar aquestes operacions.

El Cheat Sheet del DOM

El teu mapa de referència ràpida

Q1: Seleccionar

- `getElementById`
- `querySelector`
- `querySelectorAll`

Q2: Modificar

- `.textContent`
- `.innerHTML`
- `.style`
- `.classList`
- `.setAttribute`

Q3: Crea i Neteja

- `document.createElement`
- `.appendChild`
- `.insertAdjacentHTML`
- `.remove()`

Q4: Escolta

- `addEventListener`
- `event.preventDefault()`
- `event.stopPropagation()`

*Una pàgina web és un llenç. **L'HTML** és la pintura seca, però el **DOM** i **JavaScript** et converteixen en l'artista en temps real.*